

LAPORAN PRAKTIKUM
POSTTEST 7
ALGORITMA PEMROGRAMAN LANJUT



Disusun oleh:
Muhammad Adrian (2509106032)
Kelas (A2 '25)

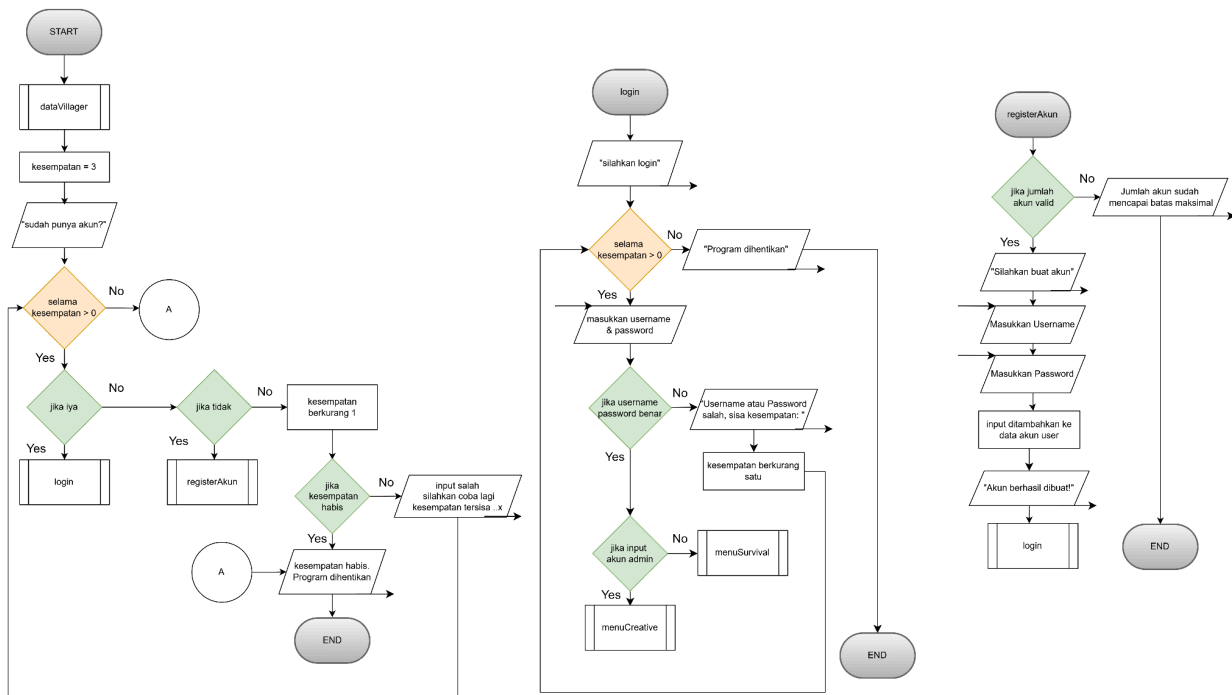
PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA

2026

1. Flowchart

Alur program yang berjalan pada program ini yaitu:

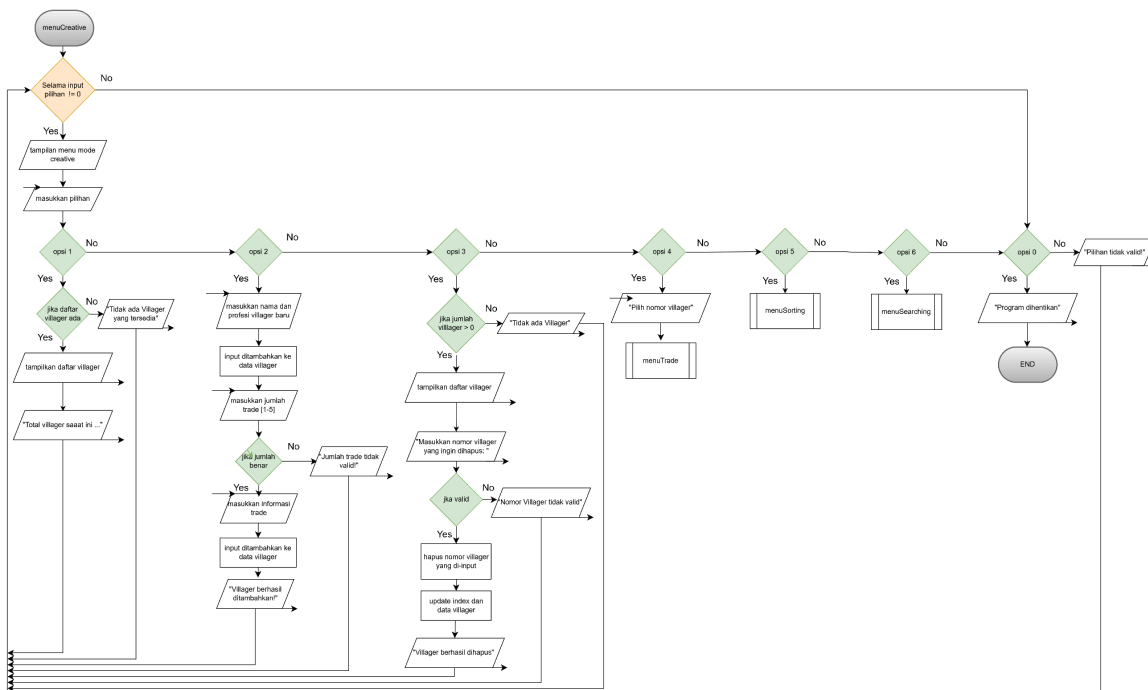
- Pada awal berjalannya program, akan ada beberapa deklarasi variabel *string*, *integer* serta variabel data dari *struct* yang berjalan dilatar belakang program.
- Program akan mulai dengan *output* dan percabangan untuk menanyakan apakah sudah punya akun atau belum.
- Jika sudah punya, pengguna akan diarahkan langsung menuju proses login dan bisa login diantara akun admin maupun user biasa.
- Jika memilih belum punya, pengguna akan diarahkan ke percabangan dimana akan diprogramkan untuk membuat atau register akun terlebih dahulu, jika sudah akan langsung diarahkan ke proses login user biasa.



Gambar 1.1 Alur Login Multiuser

Selanjutnya adalah alur program pada menu admin, atau pada studi kasus program di post test kali ini bisa disebut sebagai mode *Creative*. Adapun alur programnya yaitu:

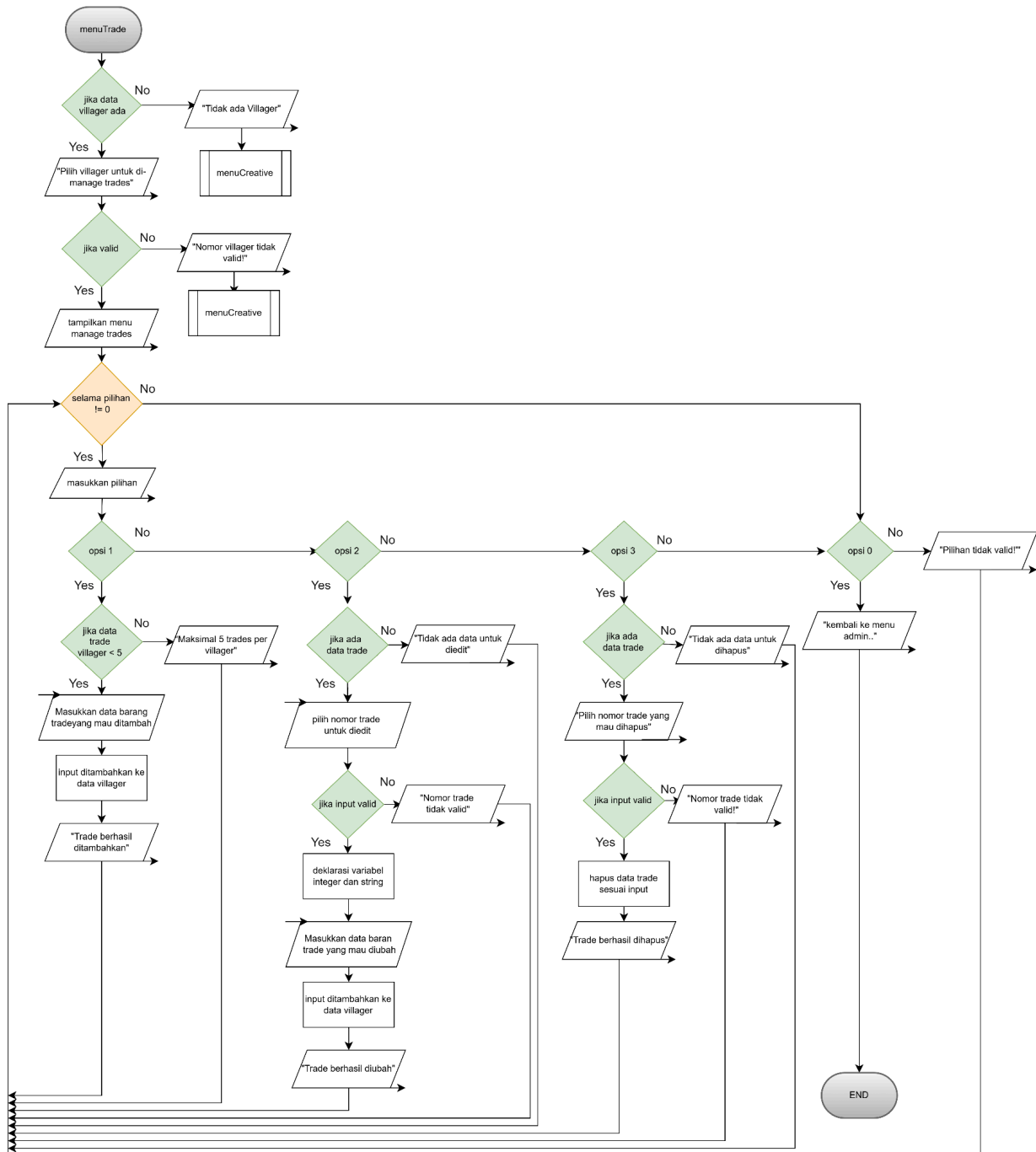
- Di awal alur menu ini akan diberikan *loop-while* agar program bisa terus berulang sampai pengguna memilih untuk keluar atau berhenti.
- Selanjutnya saat berjalan, program akan menampilkan tabel menu mode *Creative* beserta opsi-opsi pilihannya.
- Pada pilihan pertama terdapat sebuah program *Read* dengan fungsi melihat daftar *villager* yang sudah dideklarasikan maupun yang akan di ubah nanti serta memberi output jumlah villager yang tersedia dengan fungsi *tampilkanJumlah*.
- Pada pilihan kedua terdapat sebuah program *Create* dengan fungsi akan menambahkan *Villager* di daftar vilager.
- Pada pilihan ketiga terdapat program *Delete* yang akan berjalan dengan meng-input nomor *villager* lalu menghapusnya.
- Pilihan keempat adalah program manajemen trade yang akan dijelaskan di *flowchart* terlampir berikutnya.
- Opsi terakhir yaitu opsi untuk keluar atau langsung memberhentikan program secara paksa.



Gambar 1.2 Alur Menu Admin atau Creative

Selanjutnya, alur *flowchart* terlampir adalah manajemen trade dimana admin bisa menambah, merubah, dan menghapus data trade yang ada pada tiap *Villager* yang tersedia. Adapun alur programnya yaitu:

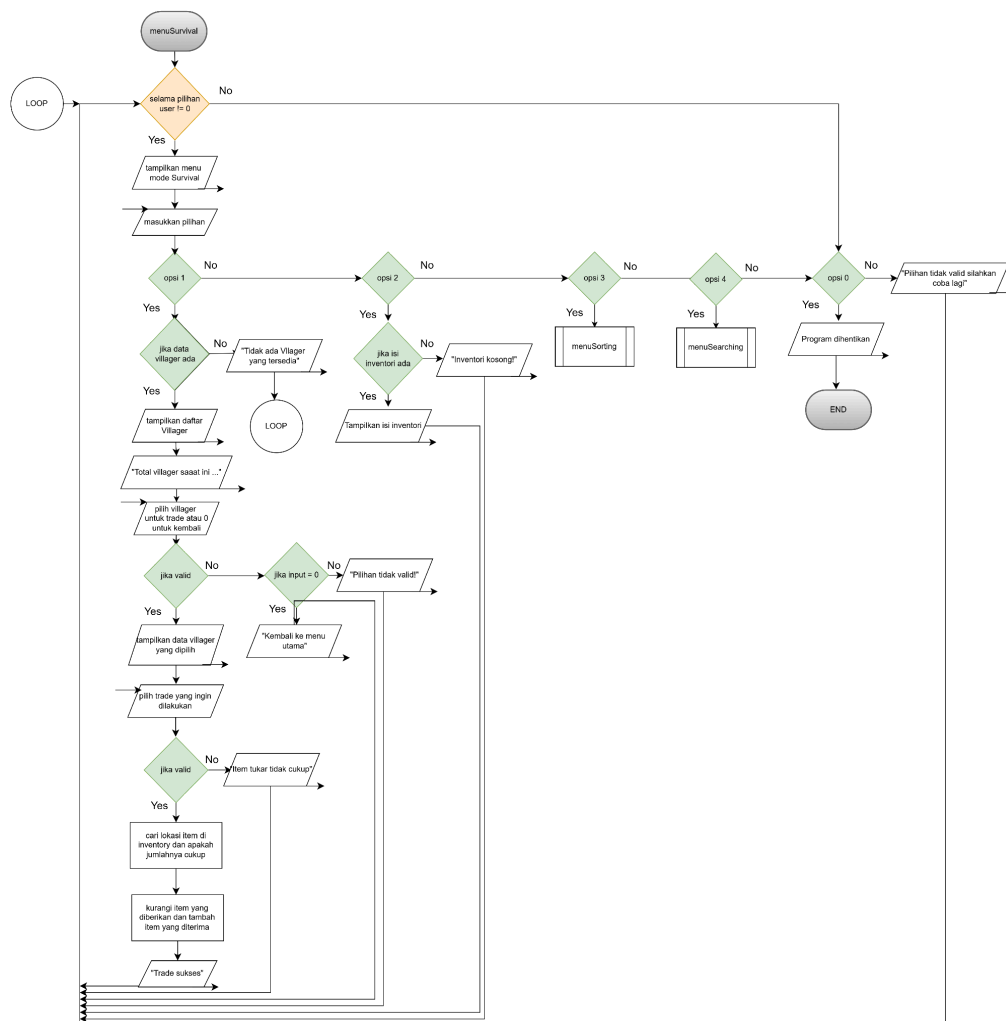
- Diawal program terdapat percabangan yang akan memastikan apakah data *villager* ada atau tidak, jika ada program akan lanjut ke tahap selanjutnya dan jika tidak ada, program akan menampilkan output “Tidak ada *Villager*” dan kembali ke menu mode *Creative*.
- Lanjut ke alur program, pengguna akan diminta untuk memilih *Villager* yang akan diedit data tradenya, jika input valid, program akan terus lanjut dan jika tidak program akan menampilkan output “Nomor *Villager* tidak valid” dan kemabli ke menu mode *Creative*.
- Selanjutnya pada alur valid, program akan menampilkan menu manajemen *trade* beserta opsi-opsi pilihan sama seperti menu mode *Creative* sebelumnya.
- Pada pilihan pertama, akan terdapat percabangan untuk memastikan apakah data *trade* masih dibawah 5 (karena 5 adalah batas data tarde per *Villager*), jika masih dibawah 5 maka program lanjut untuk menyuruh pengguna menginputdata *trade* yang mau ditambah.
- Pada pilihan kedua sama seperti sebelumnya, diawal program akan ada percabangan untuk memastikan apakah ada data *trade* yang akan diedit, jika masih ada, pengguna akan disuruh untuk menginput nomor data *trade* yang nantinya akan diubah sesuai kemauan pengguna.
- Pada pilihan ketiga juga terdapat percabangan untuk memastikan bahwa ada data *trade* yang tersedia, jika ada, pengguna akan disuruh untuk input nomor barang atau data *trade* yang mau dihapus.
- Opsi terakhir yaitu kembali, pengguna bisa kembali ke menu mode *Creative* dengan opsi ini.



Gambar 1.3 Program Menu Trade Admin atau Creative

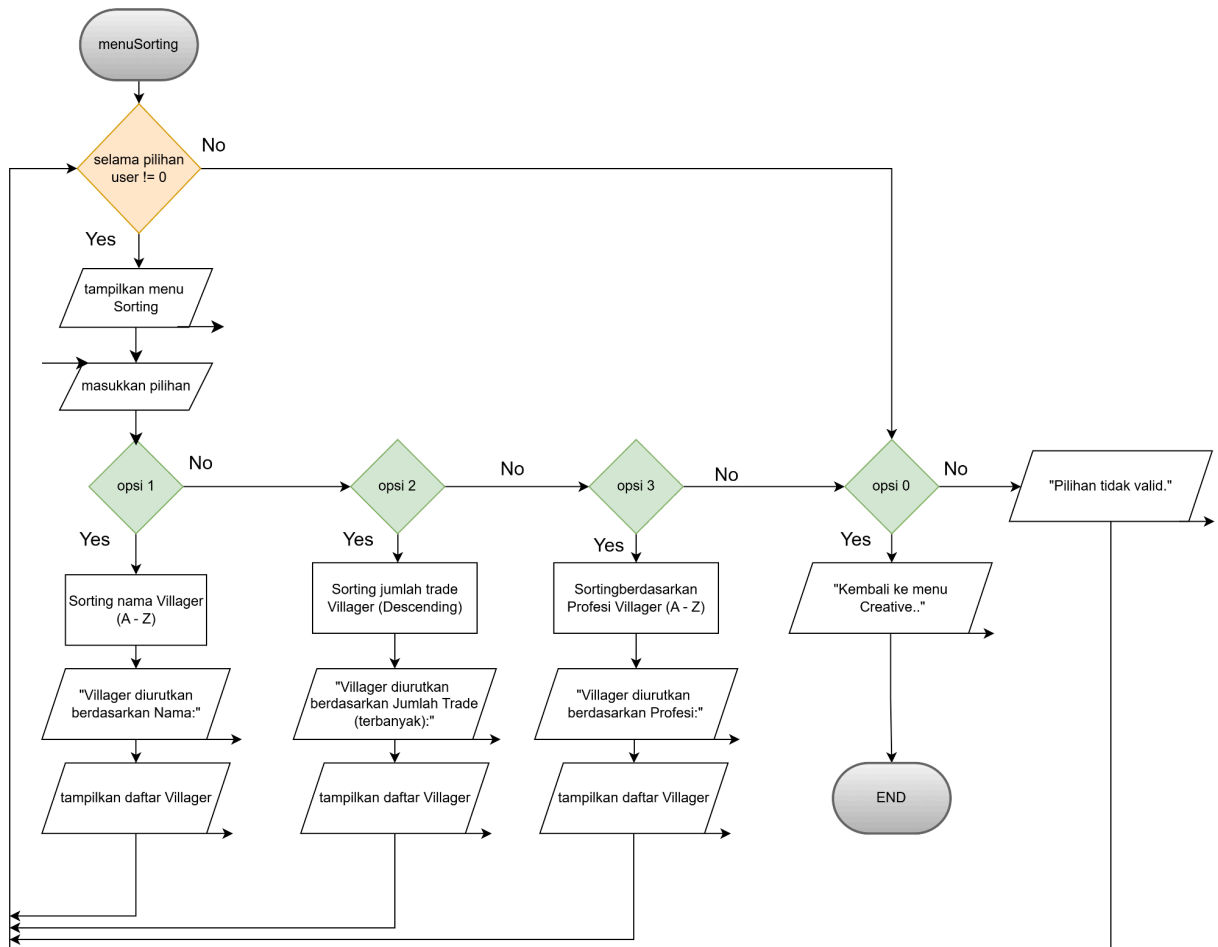
Selanjutnya, terdapat *flowchart* program menu mode *Survival* atau bisa kita sebut sebagai mode user biasa. Adapun alur pemrogramannya yaitu:

- Pada awal alur mode ini terdapat sebuah *loop-while* yang memiliki fungsi sama seperti *loop-while* sebelumnya yaitu untuk melakukan perulangan selama program berlangsung dan akan berhenti saat pengguna memilih untuk berhenti atau keluar.
- Selanjutnya program ini akan menampilkan menu mode *Survival* dengan beberapa opsi-opsi pilihan, dan pengguna disuruh input mau ke opsi berapa.
- Pada pilihan pertama diawal akan terdapat percabangan untuk memastikan jika data *villager* ada atau tidak. Jika tidak ada akan ditampilkan output “Tidak ada *Villager* yang tersedia” dan program melakukan loop dan menampilkan menu mode *Survival* kembali. Jika *Villager* ada program akan menampilkan daftar *Villager* yang tersedia dan bisa melakukan trade dengan *Villager* yang dipilih.
- Pada pilihan kedua pengguna hanya akan diperlihatkan isi inventori yang terdapat beberapa item telah dideklarasikan maupun sudah terjadi perubahan ketika sebelumnya sudah melakukan trading dengan *Villager*.
- Opsi terakhir yaitu keluar, pengguna bisa memilih opsi ini untuk keluar atau menghentikan program dengan paksa.



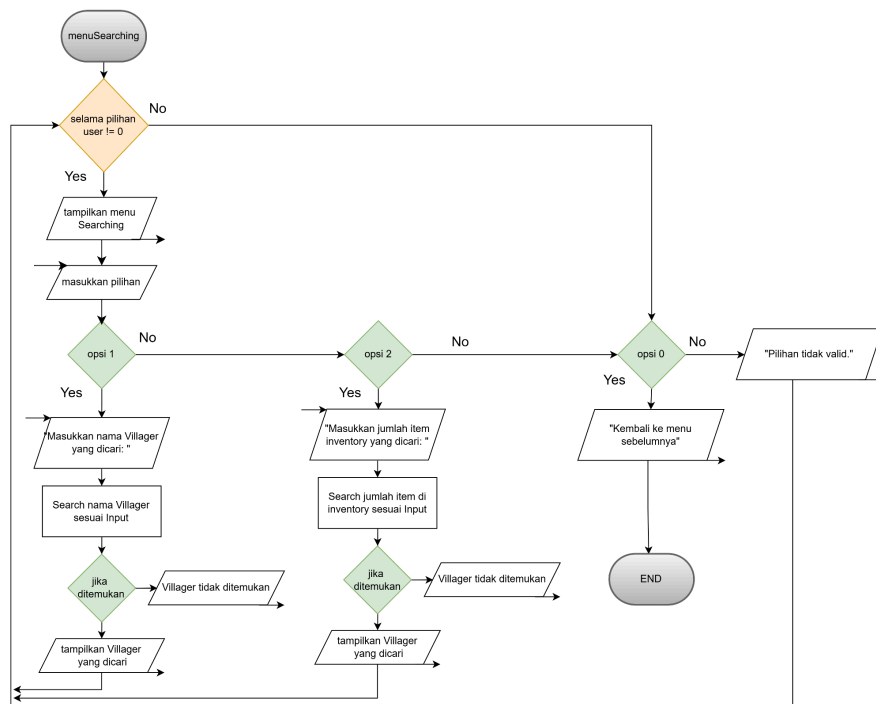
Gambar 1.4 Menu User atau Survival

Selanjutnya adalah program atau opsi terbaru pada pilihan menu *Creative* maupun *Survival*. *Flowchart* yang terlampir di bawah ini merupakan program kecil dari sebuah fungsi bernama Menu Sorting. *Admin* maupun *User* bisa mengakses *Menu Sorting* tersebut serta opsi-opsi pilihan yang ada pada menunya. Opsi pilihan pertama berfungsi untuk menyortir tampilan daftar *Villager* berdasarkan nama *Villager* itu sendiri dari huruf A hingga Z. Selanjutnya pada pilihan kedua ini berfungsi untuk menyortir berdasarkan jumlah trade yang ada pada masing masing *Villager*. Pilihan ketiga berfungsi untuk menyortir tampilan daftar *Villager* berdasarkan profesi yang dimiliki masing-masing *Villager*. Lalu seperti menu lainnya, pengguna bisa memilih menu keluar atau kembali ke menu sebelumnya.



Gambar 1.5 Menu Sorting

Terakhir adalah program atau opsi terbaru pada pilihan menu *Creative* maupun *Survival*. *Flowchart* yang terlampir di bawah ini merupakan program kecil dari sebuah fungsi bernama *Menu Searching*. *Admin* maupun *User* bisa mengakses *Menu Searching* tersebut serta opsi-opsi pilihan yang ada pada menunya. Opsi pilihan pertama berfungsi untuk menginput nama *Villager* yang ingin dicari lalu menampilkan daftar atau profil *Villager* yang dicari. Selanjutnya pada pilihan kedua ini berfungsi untuk menginput jumlah item di inventory yang ingin dicari.. Lalu seperti menu lainnya, pengguna bisa memilih menu keluar atau kembali ke menu sebelumnya.



Gambar 1.6 Menu Searching

2. Deskripsi Singkat Program

Program berjudul *Minecraft Villager Trading Hall* ini berbasis bahasa pemrograman C++ yang mensimulasikan sistem perdagangan *villager* di game *Minecraft*, menggunakan *struct*, untuk mengelola akun pengguna, data *villager*, inventori dan sistem *trade* yang realistis. Adapun fungsi atau manfaat utama dari program ini yaitu untuk memahami konsep pemrograman *struct*, *array*, dan *CRUD operations* dengan implementasi studi kasus sederhana dari tema program ini.

3. Source Code

3.1. Menu Trade

```
void menuTrade(){
    int vill_index;
    if (jumlah_villager > 0) {
        cout << " [>] Pilih nomor villager untuk di-manage trades [1-" <<
jumlah_villager << "]: ";
        try {
            cin >> vill_index;
            if (cin.fail()){
                cin.clear();
                cin.ignore(1000, '\n');
                throw runtime_error(" [!] Input tidak valid. Kembali ke menu
sebelumnya. ");
            }
            if (vill_index >= 1 && vill_index <= jumlah_villager) {
                int pilihan_trade;
                do {
                    cout << endl;
                    cout << ">>>=====<<<" <<
endl;
                    cout << ">>> Manage Trades untuk " <<
villagers[vill_index-1].nama << ": <<<" << endl;
                    cout << ">>>=====<<<" <<
endl;
                    cout << "||| 1. Tambah Trade <<<" <<
endl;
                    cout << "||| 2. Edit Trade <<<" <<
endl;
                    cout << "||| 3. Hapus Trade <<<" <<
```

```

endl;
        cout << "||| 4. Kembali                                     <<<" <<
endl;
        cout << ">>>=====<<<" <<
endl;
        cout << " [>] Pilihan: "; cin >> pilihan_trade;
        switch (pilihan_trade) {
        case 1:
            if (villagers[vill_index-1].num_trade < 5) {
                int beri_jmlh, terima_q;
                string beri_item, terima_item;
                cout << " [>] Masukkan jumlah barang yang
diberikan: "; cin >> beri_jmlh; cin.ignore();
                cout << " [>] Masukkan nama barang yang diberikan:
"; getline(cin, beri_item);
                cout << " [>] Masukkan jumlah barang yang diterima:
"; cin >> terima_q; cin.ignore();
                cout << " [>] Masukkan nama barang yang diterima:
"; getline(cin, terima_item);

                villagers[vill_index-1].trades[villagers[vill_index-1].num_trade].jumlah_diberi
= beri_jmlh;

                villagers[vill_index-1].trades[villagers[vill_index-1].num_trade].item_diberi =
beri_item;

                villagers[vill_index-1].trades[villagers[vill_index-1].num_trade].jumlah_terima
= terima_q;

                villagers[vill_index-1].trades[villagers[vill_index-1].num_trade].item_terima =
terima_item;

                villagers[vill_index-1].num_trade++;
                cout << " [^] Trade berhasil ditambahkan." << endl;
            } else {
                cout << " [!] Maksimal 5 trades per villager." <<
endl;
            }
            break;
        case 2:
            if (villagers[vill_index-1].num_trade > 0) {
                cout << "> Pilih nomor trade untuk edit [1-" <<
villagers[vill_index-1].num_trade << "]: ";
                int trade_id;
                try {

```

```

        cin >> trade_id;
        if (cin.fail()) {
            cin.clear();
            cin.ignore(1000, '\n');
            throw runtime_error(" [!] Input tidak
valid. kembali ke menu sebelumnya..");
        }
        if (trade_id >= 1 && trade_id <=
villagers[vill_index-1].num_trade) {
            int beri_jmlh, terima_q;
            string beri_item, terima_item;
            cout << " [>] Masukkan jumlah barang yang
diberikan: "; cin >> beri_jmlh; cin.ignore();
            cout << " [>] Masukkan nama barang yang
diberikan: "; getline(cin, beri_item);
            cout << " [>] Masukkan jumlah barang yang
diterima: "; cin >> terima_q; cin.ignore();
            cout << " [>] Masukkan nama barang yang
diterima: "; getline(cin, terima_item);

villagers[vill_index-1].trades[trade_id-1].jumlah_diberi = beri_jmlh;

villagers[vill_index-1].trades[trade_id-1].item_diberi = beri_item;

villagers[vill_index-1].trades[trade_id-1].jumlah_terima = terima_q;

villagers[vill_index-1].trades[trade_id-1].item_terima = terima_item;
            cout << " [^] Trade berhasil diubah." <<
endl;
        } else {
            cout << " [!] Nomor trade tidak valid. " <<
endl;
        }
    } catch ( const runtime_error& e) {
        cout << e.what() << endl;
    }
} else {
    cout << " [!] Tidak ada trade untuk diedit. " <<
endl;
}
break;
case 3:
    if (villagers[vill_index-1].num_trade > 0) {

```

```

        cout << " [>] Pilih nomor trade untuk hapus [1-" <<
villagers[vill_index-1].num_trade << "]: ";
        try {
            int trade_id;
            cin >> trade_id;
            if (cin.fail()) {
                cin.clear();
                cin.ignore(1000, '\n');
                throw runtime_error(" [!] Input tidak
valid. kembali ke menu sebelumnya..");
            }
            if (trade_id >= 1 && trade_id <=
villagers[vill_index-1].num_trade) {
                for(int k = trade_id-1; k <
villagers[vill_index-1].num_trade-1; k++){
                    villagers[vill_index-1].trades[k] =
villagers[vill_index-1].trades[k+1];
                }
                villagers[vill_index-1].num_trade--;
                cout << " [^] Trade berhasil dihapus." <<
endl;
            } else {
                cout << " [!] Nomor trade tidak valid. " <<
endl;
            }
        } catch (const runtime_error& e){
            cout << e.what() << endl;
        }
    } else {
        cout << " [!] Tidak ada trade untuk dihapus. " <<
endl;
    }
    break;
case 0:
    cout << " {^} Kembali ke menu admin.." << endl;
    break;
default:
    cout << " [!] Pilihan tidak valid." << endl;
}
} while (pilihan_trade != 0);
} else {
    cout << " [!] Nomor villager tidak valid. " << endl;
}
} catch (const runtime_error& e) {
    cout << e.what() << endl;
}

```

```

    }
} else {
    cout << " [!] Tidak ada Villager untuk manage trades.!!" << endl;
}
}

```

3.2. Menu Creative

```

void menuCreative(){
    dataVillager(jumlah_akun, jumlah_villager, jumlah_inventory);
    int pilihan_menu_admin;
    cout << " [^] Mode Creative aktif!" << endl << endl;
    do {
        cout << endl;
        cout << ">>>=====<<<" << endl;
        cout << ">>>      Menu mode Creative:      <<<" << endl;
        cout << ">>>=====<<<" << endl;
        cout << "||| 1. Lihat daftar Villager      |||" << endl;
        cout << "||| 2. Tambah Villager              |||" << endl;
        cout << "||| 3. Hapus Villager               |||" << endl;
        cout << "||| 4. Manajemen Trade              |||" << endl;
        cout << "||| 5. Sorting Villager             |||" << endl;
        cout << "||| 6. Searching                    |||" << endl;
        cout << "||| 0. Keluar                       |||" << endl;
        cout << ">>>=====<<<" << endl;
        cout << " [>] Pilihan: "; cin >> pilihan_menu_admin;
        switch (pilihan_menu_admin) {
            case 1:
                tampilkanVillager();
                break;
            case 2:
                if (jumlah_villager < MAX_VILLAGER) {
                    string nama_baru, profesi_baru, trade_baru;
                    cout << " [>] Masukkan nama villager baru: "; cin.ignore();
                    getline(cin, nama_baru);
                    cout << " [>] Masukkan profesi villager: "; getline(cin,
                    profesi_baru);

                    villagers[jumlah_villager].nama = nama_baru;
                    villagers[jumlah_villager].profesi = profesi_baru;
                    villagers[jumlah_villager].num_trade = 0;
                    try {
                        int jumlah_trade_baru;
                        cout << " [>] Berapa trades awal yang ingin ditambahkan

```

```

[1-5]? "; cin >> jumlah_trade_baru;

        if (cin.fail()){
            cin.clear();
            cin.ignore(1000, '\n');
            throw runtime_error(" [!] Input tidak valid. Trade awal
di-set ke 0. ");
            jumlah_trade_baru = 0;
        }
        if (jumlah_trade_baru > 0 && jumlah_trade_baru <= 5) {
            for(int t = 0; t < jumlah_trade_baru; t++) {
                cout << "> Masukkan trade " << t+1 << ": ";
                int jumlah_diberikan, jumlah_diterima;
                string beri_item, terima_item;
                cout << " [>] Masukkan jumlah barang yang
diberikan: "; cin >> jumlah_diberikan; cin.ignore();
                cout << " [>] Masukkan nama barang yang diberikan:
";getline(cin, beri_item);
                cout << " [>] Masukkan jumlah barang yang diterima:
";cin >> jumlah_diterima;cin.ignore();
                cout << " [>] Masukkan nama barang yang diterima:
"; getline(cin, terima_item);
                villagers[jumlah_villager].trades[t].jumlah_diberi
= jumlah_diberikan;
                villagers[jumlah_villager].trades[t].item_diberi =
beri_item;
                villagers[jumlah_villager].trades[t].jumlah_terima
= jumlah_diterima;
                villagers[jumlah_villager].trades[t].item_terima =
terima_item;
            }
            villagers[jumlah_villager].num_trade =
jumlah_trade_baru;
        } else {
            cout << " [!] Jumlah trade tidak valid. Villager
ditambahkan tanpa trade. " << endl;
        }
    } catch (const runtime_error& e) {
        cout << e.what() << endl;
    }
    jumlah_villager++;
    cout << " [^] Villager baru ditambahkan" << endl;
} else {
    cout << " [!] Maksimal " << MAX_VILLAGER << " villagers sudah
tercapai. " << endl;
}

```

```

    }
    break;
case 3:
    tampilkanVillager();
    if (jumlah_villager > 0) {
        try {
            cout << " [>] Masukkan nomor villager yang ingin dihapus
[1-" << jumlah_villager << "]: ";
            int index; cin >> index;
            if (cin.fail()){
                cin.clear();
                cin.ignore(1000, '\n');
                throw runtime_error(" [!] Input tidak valid. Kembali ke
menu admin. ");
            }
            if (index >= 1 && index <= jumlah_villager) { // jika
valid, geser semua villager setelah index ke kiri
                for(int i = index-1; i < jumlah_villager-1; i++){
                    villagers[i] = villagers[i+1];
                }
                jumlah_villager--;
                cout << " [^] Villager berhasil dihapus." << endl;
            } else {
                cout << " [!] Nomor villager tidak valid. " << endl;
            }
        } catch (const exception& e) {
            cout << e.what() << endl;
            break;
        }
    } else {
        cout << " [!] Tidak ada Villager yang tersedia untuk dihapus. "
<< endl;
    }
    break;
case 4:
    menuTrade();
    break;
case 5:
    menuSorting();
    break;
case 6:
    menuSearching();
    break;
case 0:
    cout << " [i] Program dihentikan." << endl;

```

```
        break;
    default:
        cout << " [!] Pilihan tidak valid. Silahkan coba lagi." << endl;
    }
} while(pilihan_menu_admin != 0);
}
```

3.3. Menu Survival


```

void menuSurvival(){
    int pilihan_menu_user;
    do {
        cout << ">>>=====<<<" << endl;
        cout << ">>>  Anda masuk dalam mode Survival  <<<" << endl;
        cout << ">>>=====<<<" << endl;
        cout << "||| 1. Tampilkan daftar Villager      |||" << endl;
        cout << "||| 2. Lihat Inventory                    |||" << endl;
        cout << "||| 3. Menu Sorting                        |||" << endl;
        cout << "||| 4. Searching                          |||" << endl;
        cout << "||| 0. Keluar                             |||" << endl;
        cout << ">>>=====<<<" << endl;
        cout << " [>] Pilihan: ";
        cin >> pilihan_menu_user;

        switch (pilihan_menu_user) {
            case 1:
                if (jumlah_villager > 0) {
                    cout << ">>>=====<<<" << endl;
                    cout << ">>>  Menampilkan daftar Villager... <<<" << endl;
                    cout << ">>>=====<<<" << endl;
                    for(int i = 0; i < jumlah_villager; i++){
                        cout << i+1 << ". Nama: " << villagers[i].nama << ",
Profesi: " << villagers[i].profesi << endl;
                        cout << "  Trades:" << endl;
                        for(int j = 0; j < villagers[i].num_trade; j++){
                            cout << "    " << j+1 << ". "
<< villagers[i].trades[j].jumlah_diberi << "x "
<< villagers[i].trades[j].item_diberi << " for "
<< villagers[i].trades[j].jumlah_terima << "x "
<< villagers[i].trades[j].item_terima << endl;
                        }
                    }
                    cout << ">>>=====<<<" << endl;
                    cout << " [>] Pilih Villager untuk melakukan trade [1-" <<
jumlah_villager << " atau 0 untuk kembali]: ";

                    try{
                        cin >> vill_survival_trade;
                        if (cin.fail()){
                            cin.clear();
                            cin.ignore(1000, '\n');
                            throw runtime_error(" [!] Input tidak valid. Kembali ke
menu utama. ");
                        }
                        break;
                    }
                }
            default:
                break;
        }
    } while (pilihan_menu_user != 0);
}

```

```

    }
    if (vill_survival_trade >= 1 && vill_survival_trade <=
jumlah_villager) {
        Villager* vill_ptr = &villagers[vill_survival_trade-1];
        cout << "^ Anda memilih Villager: " <<
villagers[vill_survival_trade-1].nama << endl;
        cout << ">>>=====<<<" <<
endl;
        cout << ">>>      Trades yang tersedia:      <<<" <<
endl;
        cout << ">>>=====<<<" <<
endl;
        for(int t = 0; t < vill_ptr->num_trade; t++){
            auto &tr = vill_ptr->trades[t];
            cout << "      " << t+1 << ". "
                << tr.jumlah_diberi << "x " <<
tr.item_diberi
                << " for " << tr.jumlah_terima << "x " <<
tr.item_terima << endl;
        }
        // trade program
        int pilihan_trade;
        cout << " [>] Pilih trade yang ingin dilakukan [1-" <<
vill_ptr->num_trade << "]: ";
        try{
            cin >> pilihan_trade;
            if (cin.fail()){
                cin.clear();
                cin.ignore(1000, '\n');
                throw runtime_error(" [!] Input tidak valid.
Kembali ke menu utama.");
            }
            break;
        }
        if(pilihan_trade >= 1 && pilihan_trade <=
vill_ptr->num_trade){
            Trade &tr = vill_ptr->trades[pilihan_trade-1];
            // melihat inventori dari item yang diberi
            int idx_dberi = -1, idx_dterima = -1;
            for(int k=0;k<jumlah_inventory;k++){
                if(inventory[k].item == tr.item_diberi)
idx_dberi = k;
                if(inventory[k].item == tr.item_terima)
idx_dterima = k;
            }
            if(idx_dberi != -1 &&

```

```

inventory[idx_dberi].jumlah >= tr.jumlah_diberi){
    inventory[idx_dberi].jumlah -=
tr.jumlah_diberi;

    if(idx_dterima != -1){
        inventory[idx_dterima].jumlah +=
tr.jumlah_terima;

    } else if(jumlah_inventory <
MAX_INVENTORY){
        inventory[jumlah_inventory].item =
tr.item_terima;

        inventory[jumlah_inventory].jumlah =
tr.jumlah_terima;

        jumlah_inventory++;
    }
    cout << " [^] Trade sukses: " <<
tr.jumlah_diberi << "x " << tr.item_diberi
        << " ditukar menjadi " <<
tr.jumlah_terima << "x " << tr.item_terima << "\n";
    } else {
        cout << " [!] Anda tidak punya cukup " <<
tr.item_diberi << " untuk melakukan trade." << endl;
    }
    } else {
        cout << " [!] Pilihan trade tidak valid. !" <<
endl;

    }
    }catch (const runtime_error& e) {
        cout << e.what() << endl;
    }
    } else if (vill_survival_trade == 0) {
        cout << " [!] Kembali ke menu utama.. " << endl;
    } else {
        cout << " [!] Pilihan tidak valid. " << endl;
    }
    }catch (const exception& e) {
        cout << e.what() << endl;
    }
    } else {
        cout << " [!] Tidak ada Villager yang tersedia. " << endl;
    }
    break;
case 2:
    cout << ">>>=====<<<" << endl;
    cout << ">>>          Menampilkan Inventory          <<<" << endl;
    cout << ">>>=====<<<" << endl;

```

```

        if (jumlah_inventory > 0) {
            for(int i = 0; i < jumlah_inventory; i++){
                cout << i+1 << ". " << inventory[i].item << " - Jumlah: "
<< inventory[i].jumlah << endl;
            }
        } else {
            cout << " [!] Inventory kosong. " << endl;
        }
        break;
    case 3:
        menuSorting();
        break;
    case 4:
        menuSearching();
        break;
    case 0:
        cout << " [^] Keluar dari menu Survival." << endl;
        cout << " [!] Program dihentikan." << endl;
        break;
    default:
        cout << " [!] Pilihan tidak valid. Silahkan coba lagi. " << endl;
    }
} while(pilihan_menu_user != 0);
}

```

3.4. Login

```

int login(int kesempatan){
    cout << "^ Silahkan login terlebih dahulu.." << endl;
    while(kesempatan > 0){
        cout << "> Masukkan Username: "; cin >> username;
        cout << "> Masukkan Password: "; cin >> password;
        bool login_berhasil = false;
        for(int i = 0; i < jumlah_akun; i++){
            if(username == daftar_akun[i].username && password ==
daftar_akun[i].password){
                login_berhasil = true;
                if (username == "Adrian" && password == "032"){
                    menuCreative();
                }else{
                    menuSurvival();
                }
            }
            return 1;
        }
    }
    if(!login_berhasil){

```

```

        kesempatan--;
        if(kesempatan > 0){
            cout << "!! Username atau Password salah! Kesempatan tersisa: " <<
kesempatan << "x !!" << endl;
            return login(kesempatan);
        } else {
            cout << "!! Kesempatan habis. Program dihentikan secara paksa !!"
<< endl;
            return 0;
        }
    }
    break;
}

return 0;
}

```

3.5. Register

```

int registerAkun(int kesempatan2){
    if(jumlah_akun < MAX_AKUN){
        cout << ">>> Silahkan buat akun dengan memasukkan username dan password
yang anda inginkan: <<<" << endl;
        cout << "> Masukkan Username: "; cin >> username_baru;
        cout << "> Masukkan Password: "; cin >> password_baru;

        //Simpan ke array
        daftar_akun[jumlah_akun].username = username_baru;
        daftar_akun[jumlah_akun].password = password_baru;
        jumlah_akun++;
        cout << "^ Akun berhasil dibuat!" << endl;
        login(3);
    }else {
        cout << "!! Maaf, jumlah akun sudah mencapai batas maksimal. !!" << endl;
    }
    return 0;
}

```

3.6. Sorting

```

void urutkanNama(Villager arr[], int n) {
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-i-1; j++) {
            if (arr[j].nama > arr[j+1].nama) {
                swap(arr[j], arr[j+1]);
            }
        }
    }
}

```

```

    }
}

void urutkanTrade(Villager arr[], int n) {
    // loop tiap villager
    for (int v = 0; v < n; v++) {
        // bubble sort trades[] milik villager ke-v
        for (int i = 0; i < arr[v].num_trade - 1; i++) {
            for (int j = 0; j < arr[v].num_trade - i - 1; j++) {
                if (arr[v].trades[j].jumlah_diberi <
arr[v].trades[j+1].jumlah_diberi) {
                    swap(arr[v].trades[j], arr[v].trades[j+1]);
                }
            }
        }
    }
}

void urutkanProfesi(Villager arr[], int n) {
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-i-1; j++) {
            if (arr[j].profesi > arr[j+1].profesi) {
                swap(arr[j], arr[j+1]);
            }
        }
    }
}

```

3.7. Tampilkan Daftar Villager

```

void tampilkanVillager() {
    if (jumlah_villager > 0) {
        cout << endl;
        cout << ">>>>=====<<<<<" << endl;
        cout << ">>>>          Daftar Villager:          <<<<<" << endl;
        cout << ">>>>=====<<<<<" << endl;
        for(int i = 0; i < jumlah_villager; i++){
            cout << i+1 << ". Nama: " << villagers[i].nama << ", Profesi: " <<
villagers[i].profesi << endl;
            cout << "    Trades:" << endl;
            for(int j = 0; j < villagers[i].num_trade; j++){
                cout << "        " << j+1 << ". "
                << villagers[i].trades[j].jumlah_diberi << "x "
                << villagers[i].trades[j].item_diberi << " for "

```

```

                << villagers[i].trades[j].jumlah_terima << "x "
                << villagers[i].trades[j].item_terima << endl;
            }
        }
        cout << ">>>>=====<<<<" << endl;
        tampilkanJumlah(&jumlah_villager);
    } else {
        cout << " [!] Tidak ada Villager yang tersedia. " << endl;
    }
}
}

```

3.8. Menu Sorting

```

void menuSorting() {
    int pilihan_sort;
    do {
        cout << endl;
        cout << ">>>=====<<<" << endl;
        cout << ">>>          MENU SORTING VILLAGER          <<<" << endl;
        cout << ">>>=====<<<" << endl;
        cout << "||| 1. Urutkan Nama (A -> Z)           |||" << endl;
        cout << "||| 2. Urutkan Jumlah Trade (Descending) |||" << endl;
        cout << "||| 3. Urutkan Profesi (A -> Z)           |||" << endl;
        cout << "||| 0. Kembali ke menu Creative          |||" << endl;
        cout << ">>>=====<<<" << endl;
        cout << " [>] Pilihan: "; cin >> pilihan_sort;

        switch (pilihan_sort) {
            case 1:
                urutkanNama(villagers, jumlah_villager);
                cout << " [^] Villager diurutkan berdasarkan Nama:" << endl;
                tampilkanVillager();
                break;
            case 2:
                urutkanTrade(villagers, jumlah_villager);
                cout << " [^] Villager diurutkan berdasarkan Jumlah Trade
(terbanyak):" << endl;
                tampilkanVillager();
                break;
            case 3:
                urutkanProfesi(villagers, jumlah_villager);
                cout << " [^] Villager diurutkan berdasarkan Profesi:" << endl;
                tampilkanVillager();
                break;
            case 0:

```

```

        cout << " [^] Kembali ke menu Creative.." << endl;
        break;
    default:
        cout << " [!] Pilihan tidak valid." << endl;
    }
} while (pilihan_sort != 0);
}

```

3.9. Linear Search Nama

```

void linearSearchNama(Villager* arr, int* n) {
    string keyword;
    cout << " [>] Masukkan nama Villager yang dicari: ";
    cin.ignore();
    getline(cin, keyword);

    bool ditemukan = false;
    for (int i = 0; i < *n; i++) { // melakukan iterasi satu per satu dari awal
        if (arr[i].nama == keyword) { // bandingkan nama dengan keywordnya
            cout << " [^] Villager ditemukan di index ke-" << i << "!" << endl;
            cout << "      Nama      : " << arr[i].nama << endl;
            cout << "      Profesi   : " << arr[i].profesi << endl;
            cout << "      Trades    : " << endl;
            for (int j = 0; j < arr[i].num_trade; j++) {
                cout << "          " << j+1 << ". "
                    << arr[i].trades[j].jumlah_diberi << "x " <<
arr[i].trades[j].item_diberi
                    << " for "
                    << arr[i].trades[j].jumlah_terima << "x " <<
arr[i].trades[j].item_terima << endl;
            }
            ditemukan = true;
            break; // berhenti setelah ditemukan pertama kali
        }
    }
    if (!ditemukan) {
        cout << " [!] Villager dengan nama \"" << keyword << "\" tidak
ditemukan." << endl;
    }
}

```

3.10. Jump Search Jumlah Item Inventory

```

void jumpSearchJumlah(Inventory* arr, int* n) {
    int target;

```



```

    cout << " [>] Masukkan jumlah item inventory yang dicari: ";
    cin >> target;

    // mengurutkan sementara salinan inventory berdasarkan jumlah (syarat Jump
    Search)
    Inventory temp[MAX_INVENTORY];
    for (int i = 0; i < *n; i++) temp[i] = arr[i]; // menyalin ke array
    sementara
    for (int i = 0; i < *n - 1; i++) { // bubble sort ascending
        for (int j = 0; j < *n - i - 1; j++) {
            if (temp[j].jumlah > temp[j+1].jumlah) swap(temp[j], temp[j+1]);
        }
    }

    //proses Jump search pada array sementara yang sudah terurut
    int step = (int)sqrt((double)*n); // ukuran lompatan = sqrt(n)
    int prev = 0;

    // selama blok < target, lompat per blok
    while (temp[min(step, *n) - 1].jumlah < target) {
        prev = step;
        step += (int)sqrt((double)*n);
        if (prev >= *n) { // sudah batas array woy
            cout << " [!] Item dengan jumlah " << target << " tidak ditemukan."
<< endl;
            return;
        }
    }

    //Linear search di dalam blok yang ditemukan
    bool ditemukan = false;
    while (temp[prev].jumlah <= target && prev < *n) {
        if (temp[prev].jumlah == target) {
            cout << " [^] Item ditemukan: " << temp[prev].item
                << " - Jumlah: " << temp[prev].jumlah << endl;
            ditemukan = true;
        }
        prev++;
    }
    if (!ditemukan) {
        cout << " [!] Item dengan jumlah " << target << " tidak ditemukan." <<
endl;
    }
}

```

3.11. Menu Searching

```
void menuSearching() {
    int pilihan_search;
    do {
        cout << endl;
        cout << ">>>=====<<<" << endl;
        cout << ">>>          MENU SEARCHING          <<<" << endl;
        cout << ">>>=====<<<" << endl;
        cout << "||| 1. Cari Villager by Nama          |||" << endl;
        cout << "||| 2. Cari Inventory by Jumlah          |||" << endl;
        cout << "||| 0. Kembali          |||" << endl;
        cout << ">>>=====<<<" << endl;
        cout << " [>] Pilihan: "; cin >> pilihan_search;

        switch (pilihan_search) {
            case 1:
                linearSearchNama(villagers, &jumlah_villager);
                break;
            case 2:
                jumpSearchJumlah(inventory, &jumlah_inventory);
                break;
            case 0:
                cout << " [^] Kembali ke menu sebelumnya.." << endl;
                break;
            default:
                cout << " [!] Pilihan tidak valid." << endl;
        }
    } while (pilihan_search != 0);
}
```

3.12. Main

```
int main(){
    dataVillager(jumlah_akun, jumlah_villager, jumlah_inventory);
    int kesempatan_main = 3;
    bool logged_in = false;
    while(kesempatan_main > 0 && !logged_in){
        cout << endl << "> Apakah anda sudah punya akun?[y/n]: " << endl; cin >>
        yorn;
        if (yorn == "y" || yorn == "Y"){
            if(login(3) == 1){
                logged_in = true;
            }
        } else if (yorn == "n" || yorn == "N"){
            registerAkun(3);
        }
    }
}
```

```

        logged_in = true;
    } else {
        kesempatan_main--;
        if(kesempatan_main == 0){
            cout << "!! Kesempatan habis. Program dihentikan. !! " << endl;
        } else {
            cout << "!! Input salah. Silahkan coba lagi. !! " << endl;
            cout << "! Kesempatan tersisa: " << kesempatan_main << "x !" <<
endl;
        }
    }
}
return 0;
}

```

4. Hasil Output

```

>>>> Silahkan login terlebih dahulu <<<<
> Masukkan nama: adrian
> Masukkan NIM 3 digit terakhir: 032

>>>=====<<<
>>> Login berhasil! Selamat datang di program konversi panjang Meter, Centimeter dan Kilometer! <<<
>>>=====<<<

```

Gambar 4.1 Login Berhasil

```

[^] Silahkan login terlebih dahulu..
[>] Masukkan Username: Adrian
[>] Masukkan Password: 032
[^] Mode Creative aktif!

>>>=====<<<
>>>      Menu mode Creative:      <<<
>>>=====<<<
||| 1. Lihat daftar Villager    |||
||| 2. Tambah Villager         |||
||| 3. Hapus Villager          |||
||| 4. Manajemen Trade         |||
||| 5. Sorting Villager        |||
||| 6. Searching               |||
||| 0. Keluar                  |||
>>>=====<<<
[>] Pilihan: █

```

Gambar 4.2 Login Admin

```

> Pilihan: 1
>>>>=====<<<<
>>>>      Daftar Villager:      <<<<
>>>>=====<<<<
1. Nama: Asep, Profesi: Farmer
   Trades:
   1. 24x Wheat for 1x Emerald
   2. 12x Carrot for 1x Emerald
   3. 1x Emerald for 4x Bread
2. Nama: Hayes, Profesi: Librarian
   Trades:
   1. 17x Book for 2x Emerald
   2. 10x Paper for 1x Emerald
   3. 67x Emerald for 1x Enchanted Book of Mending
3. Nama: Budi, Profesi: Blacksmith
   Trades:
   1. 8x Iron for 1x Emerald
   2. 16x Coal for 1x Emerald
   3. 67x Emerald for 1x Diamond Pickaxe
>>>>=====<<<<

```

Gambar 4.3 Lihat Daftar Villager (Admin)

```

> Pilihan: 1
!! Tidak ada Villager yang tersedia. !!

```

Gambar 4.4 Daftar Villager Kosong

```

> Pilihan: 2
> Masukkan nama villager baru: Supraman
> Masukkan profesi villager: Fisherman
> Berapa trades awal yang ingin ditambahkan [1-5]? 2
> Masukkan trade 1: > Masukkan jumlah barang yang diberikan: 6
> Masukkan nama barang yang diberikan: Fish Cod
> Masukkan jumlah barang yang diterima: 2
> Masukkan nama barang yang diterima: Emerald
> Masukkan trade 2: > Masukkan jumlah barang yang diberikan: 1
> Masukkan nama barang yang diberikan: Emerald
> Masukkan jumlah barang yang diterima: 2
> Masukkan nama barang yang diterima: Fishing Rod
^ Villager baru berhasil ditambahkan!

```

Gambar 4.5 Tambah Villager

```

> Pilihan: 3
> Masukkan nomor villager yang ingin dihapus [1-4]: 1
^ Villager berhasil dihapus.

```

Gambar 4.6 Hapus Villager

```

> Pilihan: 4
> Pilih nomor villager untuk di-manage trades [1-3]: 1
>>=====<<<
>>> Manage Trades untuk Hayes: <<<
>>=====<<<
||| 1. Tambah Trade          <<<
||| 2. Edit Trade           <<<
||| 3. Hapus Trade          <<<
||| 4. Kembali              <<<
>>=====<<<
> Pilihan: █

```

Gambar 4.7 Menu Manajemen Trade

```

> Pilihan: 1
> Masukkan jumlah barang yang diberikan: 12
> Masukkan nama barang yang diberikan: Risoles
> Masukkan jumlah barang yang diterima: 67
> Masukkan nama barang yang diterima: Emerald
^ Trade berhasil ditambahkan.

```

Gambar 4.8 Tambah Trade

```

> Pilihan:
2
> Pilih nomor trade untuk edit [1-4]: 1
> Masukkan jumlah barang yang diberikan: 26
> Masukkan nama barang yang diberikan: Risoles
> Masukkan jumlah barang yang diterima: 6767
> Masukkan nama barang yang diterima: Emerald
^ Trade berhasil diubah.

```

Gambar 4.9 Edit Trade

```

> Pilihan: 3
> Pilih nomor trade untuk hapus [1-4]: 3
^ Trade berhasil dihapus.

```

Gambar 4.10 Hapus Trade

```

>>>>=====<<<<
>>>>      Daftar Villager:      <<<<
>>>>=====<<<<
1. Nama: Hayes, Profesi: Librarian
   Trades:
   1. 26x Risoles for 6767x Emerald
   2. 10x Paper for 1x Emerald
   3. 12x Risoles for 67x Emerald
2. Nama: Budi, Profesi: Blacksmith
   Trades:
   1. 8x Iron for 1x Emerald
   2. 16x Coal for 1x Emerald
   3. 67x Emerald for 1x Diamond Pickaxe
3. Nama: Supraman, Profesi: Fisherman
   Trades:
   1. 6x Fish Cod for 2x Emerald
   2. 1x Emerald for 2x Fishing Rod

```

Gambar 4.11 Hasil Perubahan

```

>>>=====<<<
>>>      Menu mode Creative:      <<<
>>>=====<<<
||| 1. Lihat daftar Villager      |||
||| 2. Tambah Villager           |||
||| 3. Hapus Villager            |||
||| 4. Manajemen Trade           |||
||| 5. Sorting Villager          |||
||| 6. Searching                 |||
||| 0. Keluar                    |||
>>>=====<<<
[>] Pilihan: 

```

Gambar 4.12 Pilihan Keluar

```

>>> Silahkan buat akun dengan memasukkan username dan password yang anda inginkan: <<<
> Masukkan Username: Hys
> Masukkan Password: 67
^ Akun berhasil dibuat! Silahkan login.
> Masukkan Username: Hys
Masukkan Password: 67
>>>=====<<<
>>>  Anda masuk dalam mode Survival <<<
>>>=====<<<
||| 1. Tampilkan daftar Villager  |||
||| 2. Lihat Inventory           |||
||| 3. Keluar                    |||
>>>=====<<<
> Pilihan: 

```

Gambar 4.13 Register & Login Mode Survival

```

>>>=====<<<
>>>   Menampilkan daftar Villager... <<<
>>>=====<<<
1. Nama: Asep, Profesi: Farmer
   Trades:
   1. 24x Wheat for 1x Emerald
   2. 12x Carrot for 1x Emerald
   3. 1x Emerald for 4x Bread
2. Nama: Hayes, Profesi: Librarian
   Trades:
   1. 17x Book for 2x Emerald
   2. 10x Paper for 1x Emerald
   3. 67x Emerald for 1x Enchanted Book of Mending
3. Nama: Budi, Profesi: Blacksmith
   Trades:
   1. 8x Iron for 1x Emerald
   2. 16x Coal for 1x Emerald
   3. 67x Emerald for 1x Diamond Pickaxe
>>>=====<<<

```

Gambar 4.14 Tampilkan Daftar Villager (User)

```

>>>=====<<<
> Pilih Villager untuk melakukan trade [1-3 atau 0 untuk kembali]: 1
^ Anda memilih Villager: Asep
>>>=====<<<
>>>   Trades yang tersedia:   <<<
>>>=====<<<
   1. 24x Wheat for 1x Emerald
   2. 12x Carrot for 1x Emerald
   3. 1x Emerald for 4x Bread
> Pilih trade yang ingin dilakukan [1-3]: 3
^ Trade sukses: 1x Emerald ditukar menjadi 4x Bread

```

Gambar 4.15 Trading

```

>>>=====<<<
>>>   Trades yang tersedia:   <<<
>>>=====<<<
   1. 8x Iron for 1x Emerald
   2. 16x Coal for 1x Emerald
   3. 67x Emerald for 1x Diamond Pickaxe
> Pilih trade yang ingin dilakukan [1-3]: 1
!! Anda tidak punya cukup Iron untuk melakukan trade. !!

```

Gambar 4.16 Trading Gagal

```

> Pilihan: 2
>>>=====<<<
>>>      Menampilkan Inventory      <<<
>>>=====<<<
1. Emerald - Jumlah: 9
2. Wheat - Jumlah: 64
3. Iron - Jumlah: 6
4. Coal - Jumlah: 37
5. Paper - Jumlah: 46
6. Book - Jumlah: 12
7. Bread - Jumlah: 4

```

Gambar 4.17 Lihat Inventory

```

> Pilihan: 3
^ Keluar dari menu Survival.
!! Program dihentikan. !!
PS F:\Adrian's\Semester 2\Praktikum AP\post-t

```

Gambar 4.18 Pilihan Keluar (User)

```

>>>=====<<<
>>>      MENU SORTING VILLAGER      <<<
>>>=====<<<
||| 1. Urutkan Nama (A -> Z)      |||
||| 2. Urutkan Jumlah Trade (Descending) |||
||| 3. Urutkan Profesi (A -> Z)   |||
||| 0. Kembali ke menu Creative   |||
>>>=====<<<
[>] Pilihan: 

```

Gambar 4.19 Menu Sorting


```

[>] Pilihan: 1
[^] Villager diurutkan berdasarkan Nama:

>>>>=====<<<<
>>>>      Daftar Villager:      <<<<
>>>>=====<<<<
1. Nama: Asep, Profesi: Farmer
   Trades:
   1. 24x Wheat for 1x Emerald
   2. 12x Carrot for 1x Emerald
   3. 1x Emerald for 4x Bread
2. Nama: Budi, Profesi: Blacksmith
   Trades:
   1. 8x Iron for 1x Emerald
   2. 16x Coal for 1x Emerald
   3. 67x Emerald for 1x Diamond Pickaxe
3. Nama: Hayes, Profesi: Librarian
   Trades:
   1. 17x Book for 2x Emerald
   2. 10x Paper for 1x Emerald
   3. 67x Emerald for 1x Enchanted Book of Mending
>>>>=====<<<<

[^] Total villager saat ini: 3

```

Gambar 4.20 Urutkan Nama

```

[>] Pilihan: 2
[^] Villager diurutkan berdasarkan Jumlah Trade (terbanyak):

>>>>=====<<<<
>>>>      Daftar Villager:      <<<<
>>>>=====<<<<
1. Nama: Asep, Profesi: Farmer
   Trades:
   1. 24x Wheat for 1x Emerald
   2. 12x Carrot for 1x Emerald
   3. 1x Emerald for 4x Bread
2. Nama: Budi, Profesi: Blacksmith
   Trades:
   1. 67x Emerald for 1x Diamond Pickaxe
   2. 16x Coal for 1x Emerald
   3. 8x Iron for 1x Emerald
3. Nama: Hayes, Profesi: Librarian
   Trades:
   1. 67x Emerald for 1x Enchanted Book of Mending
   2. 17x Book for 2x Emerald
   3. 10x Paper for 1x Emerald
>>>>=====<<<<

[^] Total villager saat ini: 3

```

Gambar 4.21 Urutkan Jumlah Trade

```

[>] Pilihan: 3
[^] Villager diurutkan berdasarkan Profesi:

>>>>=====<<<<
>>>>      Daftar Villager:      <<<<
>>>>=====<<<<
1. Nama: Budi, Profesi: Blacksmith
   Trades:
   1. 67x Emerald for 1x Diamond Pickaxe
   2. 16x Coal for 1x Emerald
   3. 8x Iron for 1x Emerald
2. Nama: Asep, Profesi: Farmer
   Trades:
   1. 24x Wheat for 1x Emerald
   2. 12x Carrot for 1x Emerald
   3. 1x Emerald for 4x Bread
3. Nama: Hayes, Profesi: Librarian
   Trades:
   1. 67x Emerald for 1x Enchanted Book of Mending
   2. 17x Book for 2x Emerald
   3. 10x Paper for 1x Emerald
>>>>=====<<<<

[^] Total villager saat ini: 3

```

Gambar 4.22 Urutkan Profesi

```

>>>=====<<<
>>>      MENU SEARCHING      <<<
>>>=====<<<
||| 1. Cari Villager by Nama    |||
||| 2. Cari Inventory by Jumlah |||
||| 0. Kembali                  |||
>>>=====<<<
[>] Pilihan: 

```

Gambar 4.23 Menu Searching

```

[>] Pilihan: 1
[>] Masukkan nama Villager yang dicari: Hayes
[^] Villager ditemukan di index ke-1!
   Nama   : Hayes
   Profesi : Librarian
   Trades :
   1. 17x Book for 2x Emerald
   2. 10x Paper for 1x Emerald
   3. 67x Emerald for 1x Enchanted Book of Mending

```

Gambar 4.24 Search Nama Villager

```

[>] Pilihan: 2
[>] Masukkan jumlah item inventory yang dicari: 64
[^] Item ditemukan: Wheat - Jumlah: 64

```

Gambar 4.25 Search Jumlah Item

5. Langkah-langkah GIT

5.1 GIT Add

Perintah '*git add*' berfungsi untuk menyiapkan perubahan file untuk disimpan, lalu tanda titik (.) berfungsi untuk memasukkan semua file yang ada di folder sebelum di-commit.

```
PS E:\Adrian's\Semester 2\Praktikum APL\post-test\post-test-apl-1> git add .
```

Gambar 5.1.1 Git Add

5.2 GIT Commit

Perintah '*git commit -m "pesan"*' berfungsi untuk menyimpan perubahan pada riwayat Git lokal dan menambahkan judul ataupun deskripsi sesuai yang user masukkan.

```
PS E:\Adrian's\Semester 2\Praktikum APL\post-test\post-test-apl-1> git commit -m "First commi serta penyelesaian keseluruhan program"
[main (root-commit) 22fe063] First commi serta penyelesaian keseluruhan program
5 files changed, 101 insertions(+)
create mode 100644 2509106032_MUHAMMAD_ADRIAN_PT_1.cpp
create mode 100644 2509106032_MUHAMMAD_ADRIAN_PT_1.exe
create mode 100644 tempCodeRunnerFile.cpp
create mode 100644 test.cpp
create mode 100644 test.exe
```

Gambar 5.2.1 Git Commit

5.3 GIT Push

Perintah '*git push -u origin main*' berfungsi untuk mengirimkan semua hasil commit ke repositori yang sudah dibuat sebelumnya.

```
PS E:\Adrian's\Semester 2\Praktikum APL\post-test\post-test-apl-1> git push -u origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 2 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 58.12 KiB | 179.00 KiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/a-durian/Praktikum_APL.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Gambar 5.3.1 Git Push